



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

MO52 Software Engineering

1er avril 2025 – Yazan MUALLA

Object-oriented design principles and practices



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Introduction

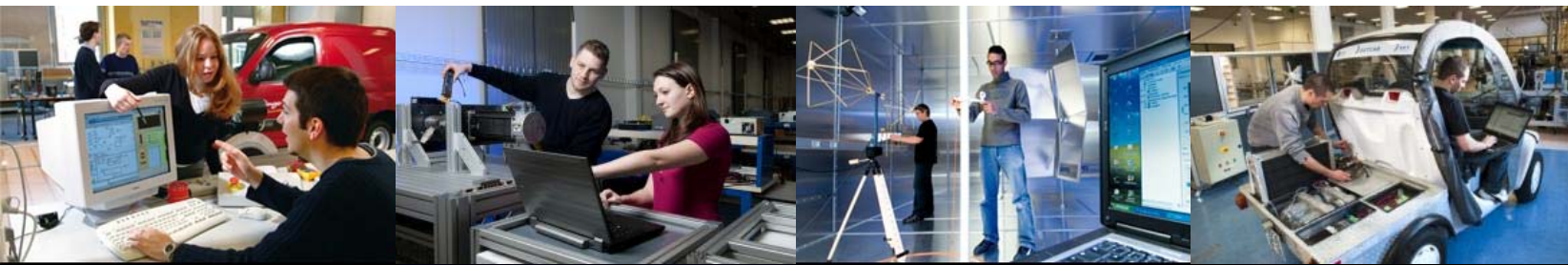
Object-oriented modelling

- When solving a problem, object-oriented modeling involves the practice of representing key concepts through objects in your software
- Objects are all around you
 - **Example of the course room:** the room itself, physical objects, people, ...
 - Specific details associated with each object
 - Individual responsibilities or behaviors

Object-oriented modelling

- Keep your code **organized, flexible** and **reusable**
- Objects are **self-aware** so they know their information

Think Objects!



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Design principles of object oriented programs



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

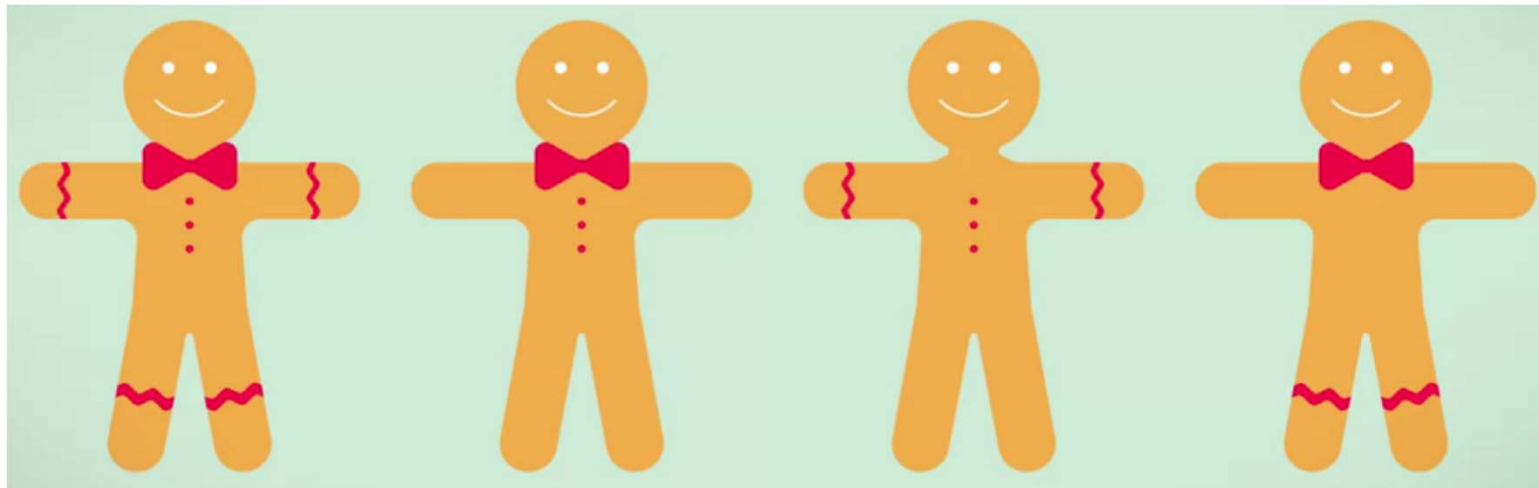
Abstraction

Abstraction

- To deal with **complexity**, abstraction is the idea of simplifying a concept in the problem domain to its essentials within some context
- Good abstraction considers the context and emphasizes the needed essentials and removes details that are not essential, example: Food?
- **Rule of Least Astonishment:** the abstraction captures the essential attributes and behavior for a concept with no surprises and no definitions that fall beyond its scope

Abstraction (cont.)

- Abstraction pertains most directly to the notion of a **class**
- With abstraction, it makes the most sense to define all of those details in a class named after the concept
- A class is like a **template for instances** of a concept



Abstraction (cont.)

- Example of an abstraction of a student:
 - **Concept's basic attributes (characteristics) of students:** the courses they take, their grades in each course and their student ID number **but not** his pet name or his height. The attributes do not disappear over time although their values may change since they are essential characteristics of a student
 - **Concept's basic behaviors (responsibilities) of students:** studying, doing assignments, and attending lectures
- Abstractions are not a fixed creation, but are a direct result of the problem for which you created them

Abstraction in Java and UML

- Developers use technical diagrams like UML Diagrams to express their designs
- A UML Class Diagram, or just Class Diagram for short, allows you to represent your design in more detail
- Class Diagrams are much closer to the implementation and can easily be converted to classes in code

Class Diagram



- **Properties**

<variable name>:<variable type>

- **Operations**

<name>(<parameter list>) : <return type>

From Class Diagram to Code



```
public class Food {  
    private String groceryID;  
    private String name;  
    private String manufacturer;  
    private Date expiryDate;  
    private double price;  
    public boolean isOnSale() {  
    }  
}
```

From Code to Class Diagram

```
public class ClickCounter {  
    private int count;  
  
    public ClickCounter() {  
        count = 0;  
    }  
  
    public void click() {  
        count++;  
    }  
    public void setClickCount(int clickCount) {  
        count = clickCount;  
    }  
  
    public int getClickCount() {  
        return count;  
    }  
}
```

From Code to Class Diagram

ClickCounter
-count: int
+click(): void
+setClickCount(clickCount: int): void
+getClickCount(): int



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Encapsulation

Encapsulation

- Bundle attributes (data), and behaviors (functions or methods), that manipulate those attributes together into a self-contained object
- Expose certain data and functions of that object through some interface, which can be accessed from other objects
- Restrict access to certain data and functions to only within that object
- **In short**, encapsulation forms a self-contained object by bundling the data and functions it requires to work, exposes an interface whereby other objects can access and use it, and restricts access to certain inside details

Encapsulation: Example

- A student object would only contain relevant data for themselves, like the degree program
- A course object would know a list of students taking it
- A professor object would know a list of courses the professor teaches
- None of these types of objects would contain a list of other courses offered, because that is not relevant data for them

Student

Course

Professor

Encapsulation

■ Black Box Thinking

- Internal details are hidden
- Provide inputs and obtain outputs by calling functions
- Reduce complexity for the users of a class
- Increases re-usability, because another class only needs to know the right method to call and the parameters to get the desired behavior

Encapsulation: Benefits

- Encapsulation helps with **data integrity**
 - Define certain attributes and methods of a class to be restricted from outside to access (Private access modifier)
 - The attribute values of an object **cannot be changed directly** through variable assignments. Otherwise, such changes could break some assumption, or dependency for the data within an object
- Encapsulation can **secure sensitive information**
 - E.g., allow a student class to store the grade point average or GPA. Then, provide some methods that use the grade without necessarily revealing it.
- Encapsulation **enhances modifiability and flexibility**
 - The accessible interface of a class can remain the same, while the implementation of the attributes and methods can change
 - Outsiders using the class, do not need to care how the implementation works behind the interface

Encapsulation in Java and UML

Student
-gpa: float -degreeProgram: String
+getGPA(): float +setGPA(float) +getDegreeProgram(): String +setDegreeProgram(String)

Encapsulation in Java and UML

- **Getter Methods** are methods that retrieve data, and their names typically begin with “get” and end with the name of the attribute whose value you will be returning
- **Setter Methods** change data, and their names typically begin with “set” and end with the name of the variable you wish to set
- Data integrity is why you have Getter and Setter Methods



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Decomposition

Decomposition

- Decomposition is taking a whole thing and dividing it up into different parts. Or, on the flip side taking a bunch of separate parts with different functionalities and combining them together to form a whole
- Decomposition allows you to further break down problems into pieces that are easier to understand and solve
- For example, a refrigerator has several parts. There is the cabinet and doors, compressor and coils, freezer, ice-maker, shelves, drawers, and of course food if the refrigerator is stocked. Each of those parts has a very specific purpose to help achieve the responsibilities of the whole

Decomposition (cont.)

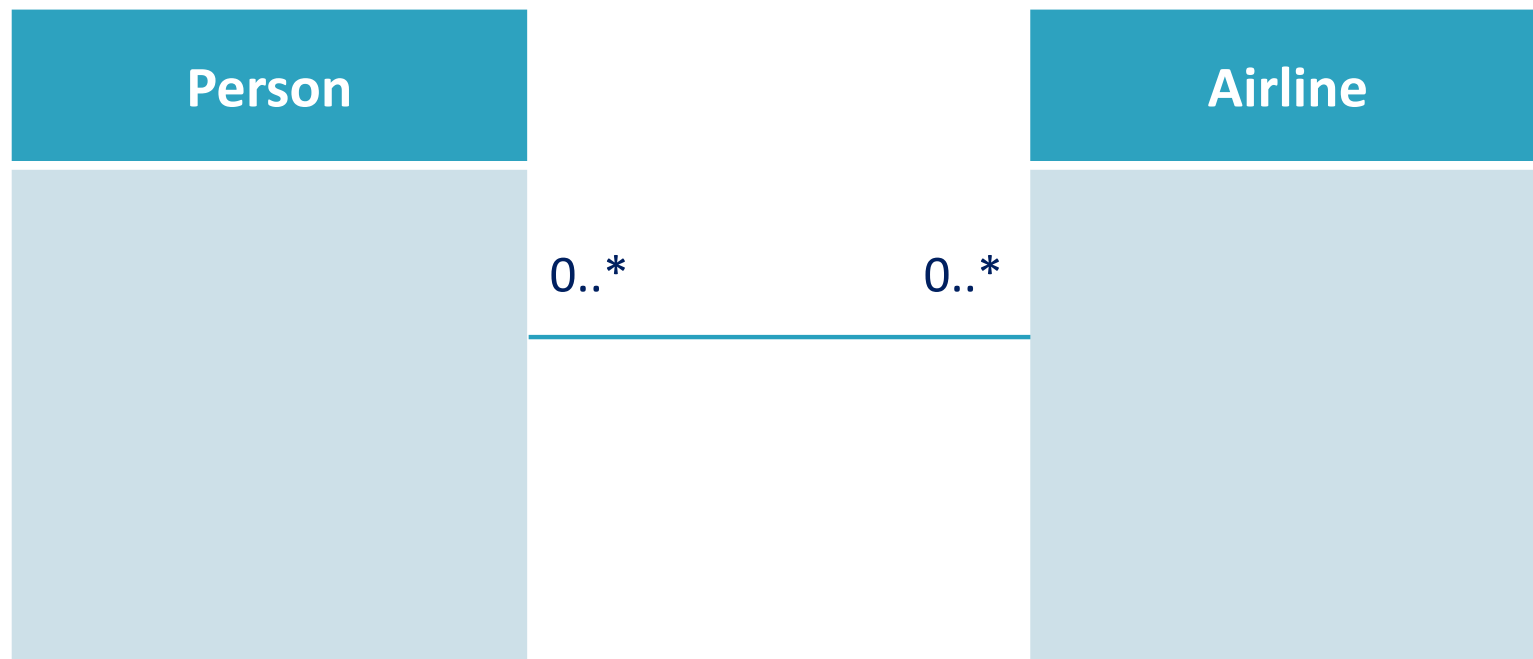
- A whole might have a **fixed** or **dynamic** number of a certain type of part
- A part itself can also serve as a whole containing further constituent parts
- One issue in decomposition involves the **lifetimes** of the whole object, and the part objects, and how they could relate.

Decomposition in Java and UML

- There are three types of relationships found in decomposition:
 - Association
 - Aggregation
 - Composition

Decomposition in UML: Association

- Association is **some** relation, a loose relationship between two objects. They may interact with each other for some time
- Example: A person does not generally own an airline, but can interact with one. An airline can also interact with many person objects. There are some persons and some airlines, neither is dependent on the other



Association vs. Dependency

- The student is passed a sport object to play. The relationship is between two completely separate objects. A student can play any number of sports. And any number of students can play a sport

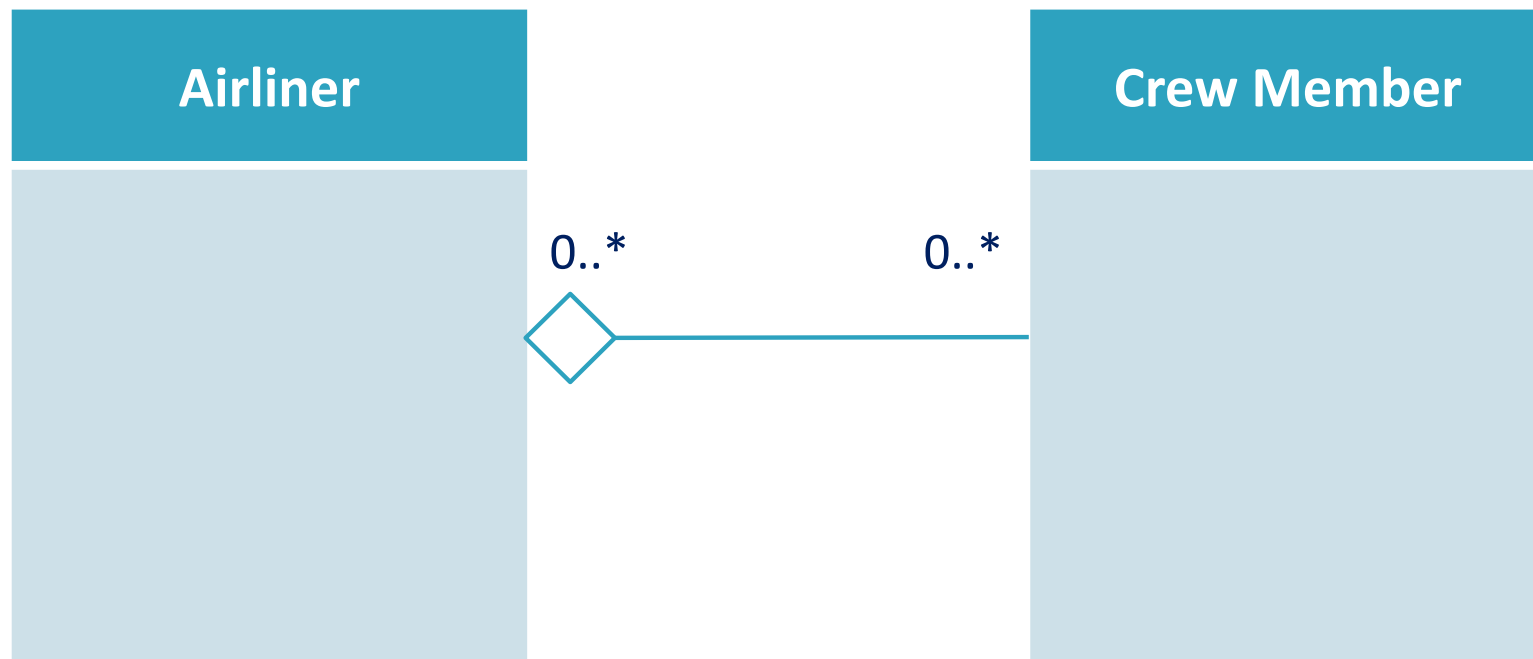
```
public class Student {  
    private Sport someSport;  
    ...  
}
```

OR

```
public class Student {  
    public void play( Sport someSport ){  
        ...  
    }  
}
```

Decomposition in UML: Aggregation

- Aggregation is a **has-a** relationship where a whole has parts that belong to it.
- The **has-a** relationship from the whole to the parts is considered **weak**. What this means is although parts can belong to the wholes, they can also exist independently



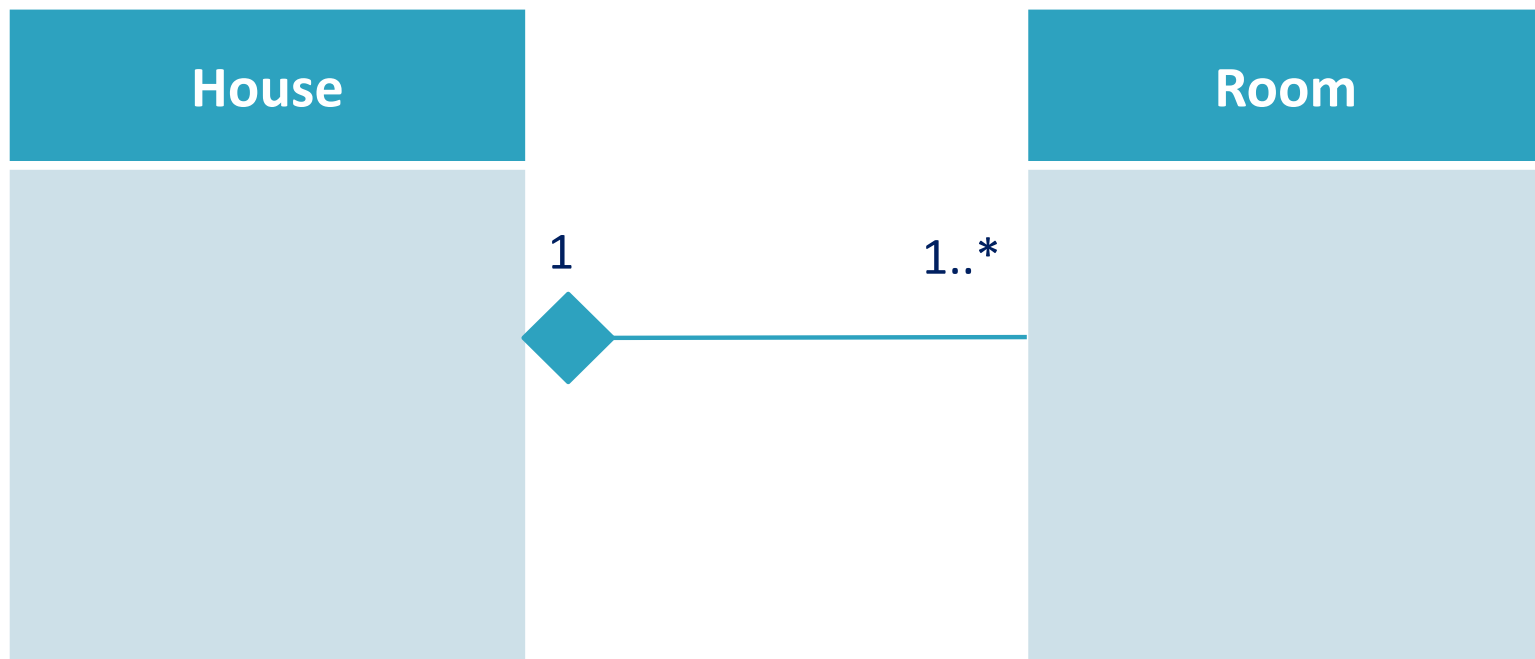
Decomposition in code: Aggregation

- In the airliner class, there is a list of crew members
- The list of crew members is initialized to be empty
- And a public method allows new crew members to be added
- An airliner can have zero or more crew members

```
public class Airliner {  
    private ArrayList<CrewMember> crew;  
  
    public Airliner() {  
        crew = new ArrayList<CrewMember>();  
    }  
  
    public void add( CrewMember crewMember ) { ...  
}  
}
```

Decomposition in UML: Composition

- Composition is an exclusive containment of parts, otherwise known as a **strong has-a** relationship
- The whole cannot exist without its parts, and the other way around
- If it loses any of its parts, the whole ceases to exist
- If the whole is destroyed, then all of its parts are destroyed too
- Usually, you can only access the parts through its whole
- Contained parts are exclusive to the whole



Decomposition in Code: Composition

- The *house* is created at the same time that the *room* object is
- The *room* does not need to be instantiated anywhere else, nor does it need to be passed into the house object on creation
- The *room* is automatically created with the *house*
- The two parts, *house* and *room*, are tightly dependent with one not being able to exist without the other

```
public class House {  
    private final ArrayList<Room> rooms;  
  
    House()  
    {  
        this.rooms = new ArrayList<Room>(...);  
    }  
}
```

Aggregation vs. Composition

```
class Car {  
  
    private Engine engine;  
  
    public void setEngine(Engine engine) {  
        this.engine = engine;  
    }  
}
```

Aggregation

```
class Car {  
  
    private final Engine engine;  
  
    Car(EngineSpecs specs) {  
        engine = new Engine(specs);  
    }  
}
```

Composition

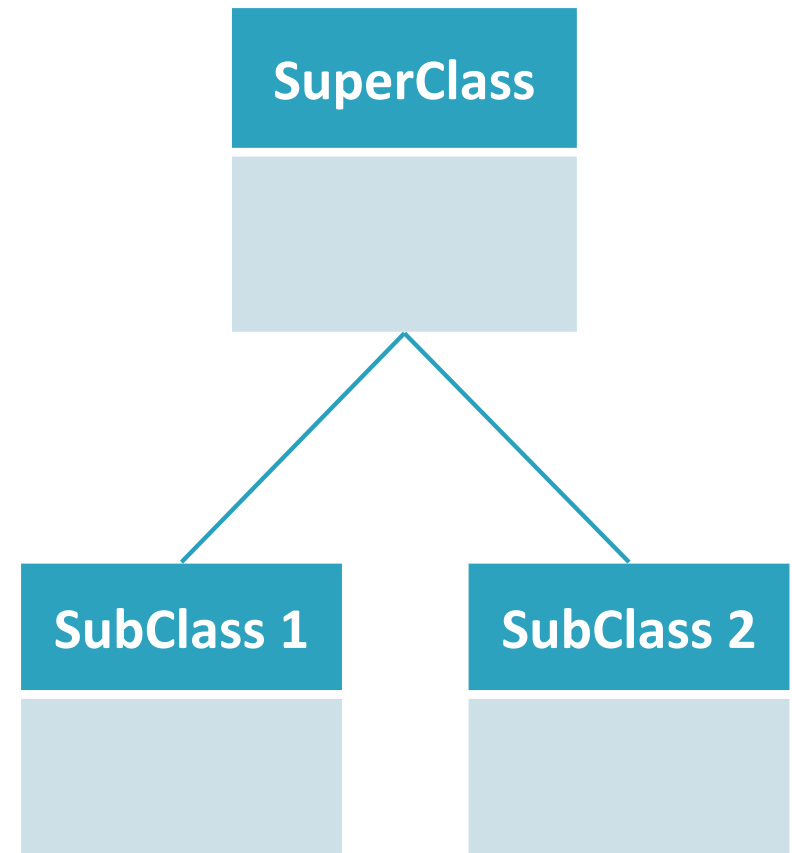


UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Generalization

Generalization

- Generalization, helps us **reduce redundancy** when solving problems
- Many behaviors and systems in the real world operate through **repetitious actions**
- Generalization can be achieved by classes through **inheritance**. In standard terminology, a *parent* class is known as a *superclass* and a *child* class is called the *subclass*



Generalization (cont.)

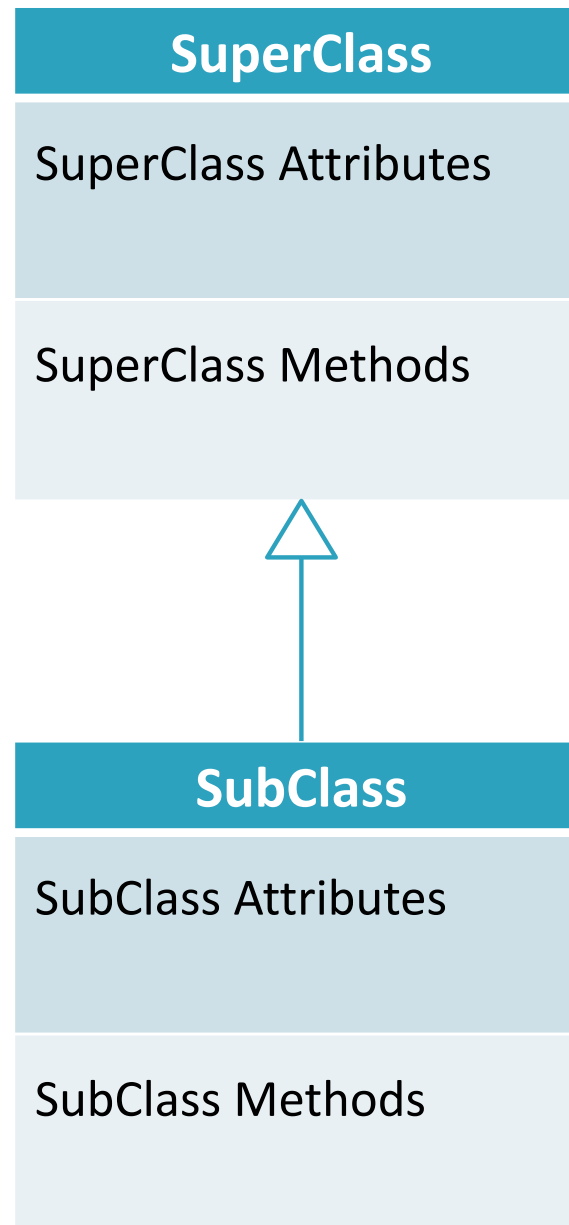
- Any changes to the code that is common to both subclasses, can be made in just a superclass
- We can easily add more classes to our system, without having to write out all the common attributes and behaviors for them
- Don't Repeat Yourself (D.R.Y.)
- It makes code more reusable because different classes or methods can share the same blocks of code
- Generalization enhances flexibility, maintainability, or reusability



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Generalization with inheritance

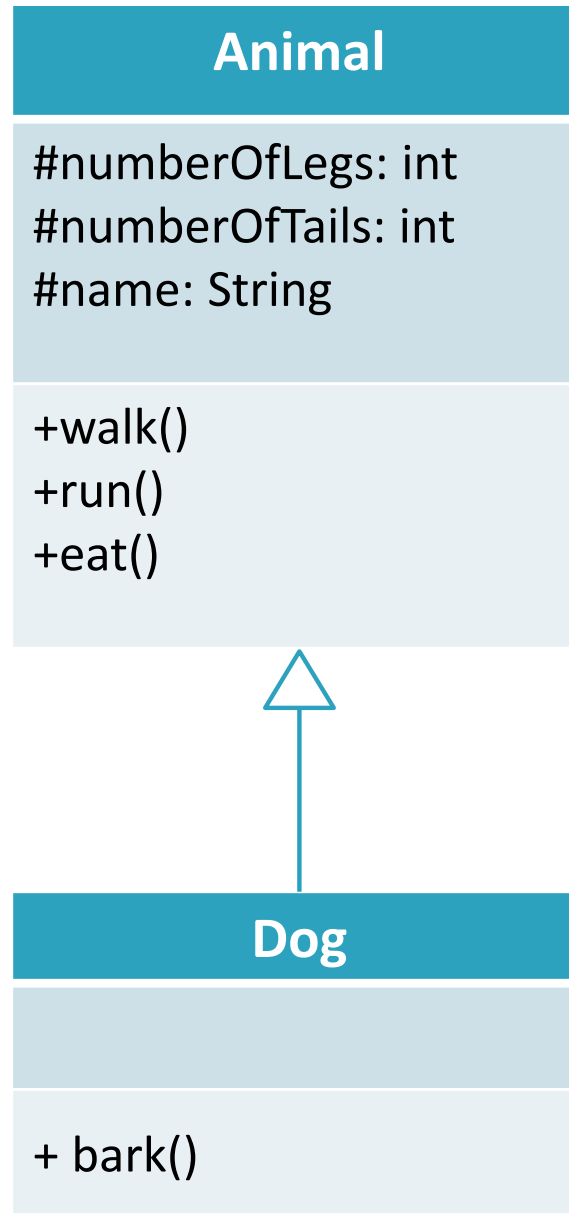
Generalization with Inheritance in UML



Generalized classes

Specialized classes

Generalization with Inheritance in UML



Generalization with Inheritance in Java

```
Public class Animal {  
    protected int numberOfLegs;  
    protected int numberOfTails;  
    protected String name;  
  
    public Animal( String petName, int legs, int tails )  
    {  
        this.name = petName;  
        this.numberOfLegs = legs;  
        this.numberOfTails = tails;  
    }  
  
    public void walk() { ... }  
    public void run() { ... }  
    public void eat() { ... }  
}
```

Generalization with Inheritance in Java

```
public class Dog extends Animal {  
    public Dog( String name, int legs, int tails) {  
        super( name, legs, tails );  
    }  
  
    public void bark() { ... }  
}
```

Implicit vs. Explicit constructors

```
public class Animal {  
    protected int numberOfLegs;  
  
    public void walk() { ... }  
}
```

Implicit or default
constructor

```
Public class Animal {  
    protected int numberOfLegs;  
  
    public Animal( int legs ) {  
        this.numberOfLegs = legs;  
    }  
    public void walk() { ... }  
}
```

Explicit
constructor



Generalization with Inheritance in Java

```
public class Animal {
    protected int numberOfLegs;

    public Animal( int legs ) {
        this.numberOfLegs = legs;
    }
    public void walk() {
        System.out.println( "Animal is walking" );
    }
}

public class Cat extends Animal {
    public Cat( int legs ) {
        super( legs );
    }
    public void walk() {
        System.out.println( "I'd rather nap" );
    }
}
```

Overload vs. Override

<i>Feature</i>	<i>Method Overload</i>	<i>Method Override</i>
Definition	Multiple methods with the same name but different parameters	Subclass redefines a method from the parent class
Where It Occurs	Same class	Parent and child classes (inheritance)
Method Signature	Must be different (different number or type of parameters)	Must be identical
Return Type	Can be different	Must be the same (or covariant)
Access Modifiers	Can be changed	Cannot reduce visibility (e.g., public cannot be overridden as private)
Polymorphism Type	Compile-time polymorphism	Runtime polymorphism
Annotation Used?	No annotation needed	@Override (recommended)

Polymorphism

```
public class Animal{
    public void speak(){
        System.out.println("Animal is making a sound");
    }
}
public class Dog extends Animal{
    public void speak(){
        System.out.println("Bark");
    }
}
public class Cat extends Animal{
    public void speak(){
        System.out.println("Meow");
    }
}
-----
public static void main(String args[]){
    Animal obj = new Dog();
    obj.speak();
}
-----
```

Output: Bark



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

Generalization with interfaces

Different types of inheritance

- The inheritance that we have been looking at is called inheritance. **For Java, only single inheritance is allowed**
- The other type of Generalization is with **Interfaces**
- A JAVA interface also denotes a type. Unlike a class, however, an interface only declares method signatures, and no constructors, attributes, or method bodies

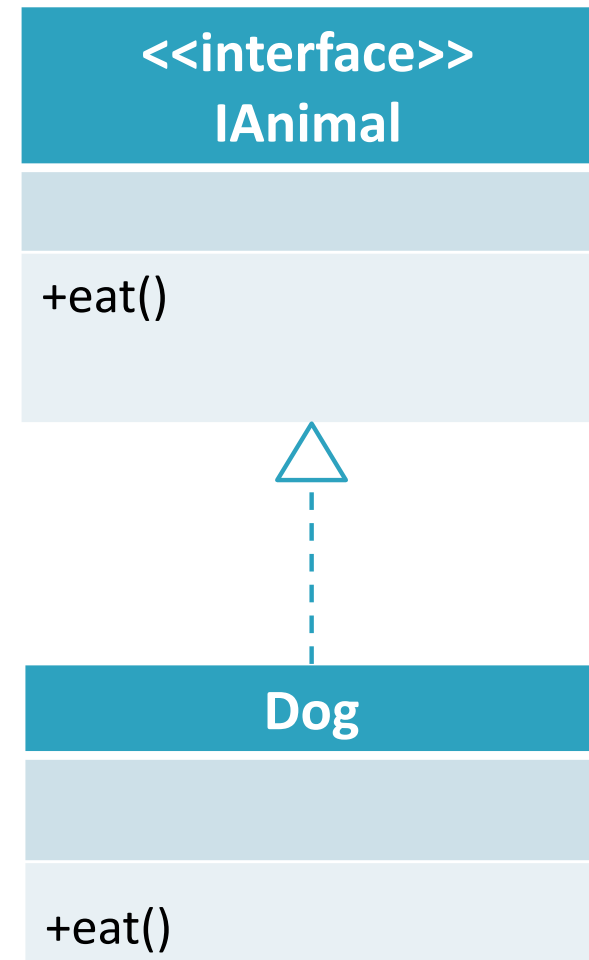
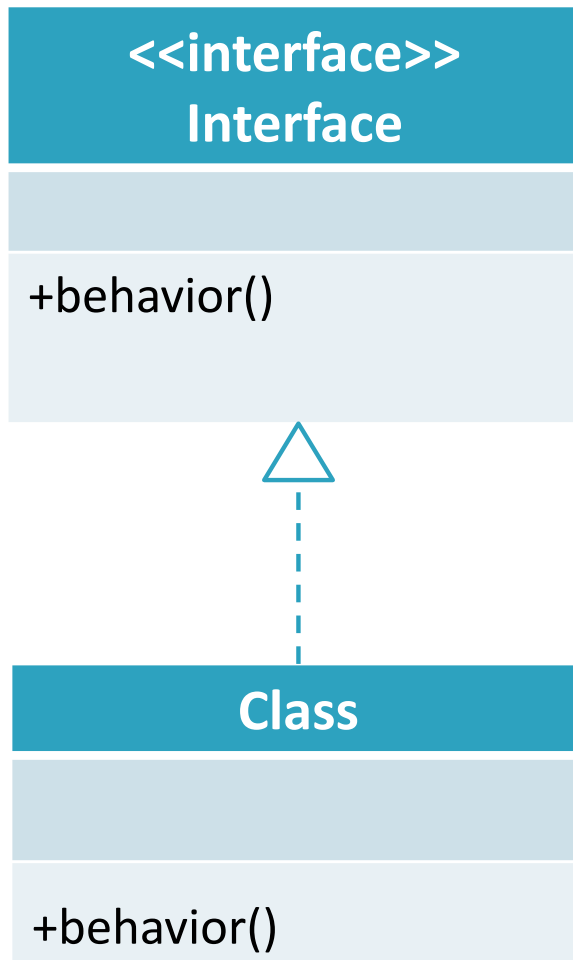
Generalization with Interfaces in Java

- A JAVA interface specifies the expected behaviors in the method signatures, but does not provide any implementation details
- In JAVA, an interface is also used for subtyping. Just like with inheritance, the dog is an animal. However, the difference is that the dog class needs to provide the implementation details for what it means to be an animal
- So, an interface is like a **contract** to be fulfilled by implementing classes

Generalization with Inheritance vs. with Interfaces

- In both inheritance and interfaces, you achieve consistency (subtyping) between the dog type and the animal type so that a dog object is usable anywhere in your program when you are dealing with an animal type
- Unlike inheritance, interfaces are not a generalization of a set of classes. It is important to understand that interfaces are not classes. They are used to describe behaviors. All that an interface contains are method signatures
- In JAVA, we use the key word **interface** to indicate that we are creating one. Standard JAVA naming convention places the letter I before an actual name to indicate an interface

Generalization with Interfaces in UML



Generalization with Interfaces in Java

```
public interface IAnimal {  
    public void move();  
    public void speak();  
    public void eat();  
}
```

```
public class Dog implements IAnimal {  
    /* Attributes of a dog can go here  
*/  
  
    public void move() { ... }  
  
    public void speak() { ... }  
  
    public void eat() { ... }  
}
```

Interface inheritance

```
public interface IVehicleMovement {  
    public void moveOnX();  
    public void moveOnY();  
}
```

```
public interface IVehicleMovement3D extends  
IVehicleMovement {  
    public void moveOnZ();  
}
```

Multiple inheritance vs. multiple implementation

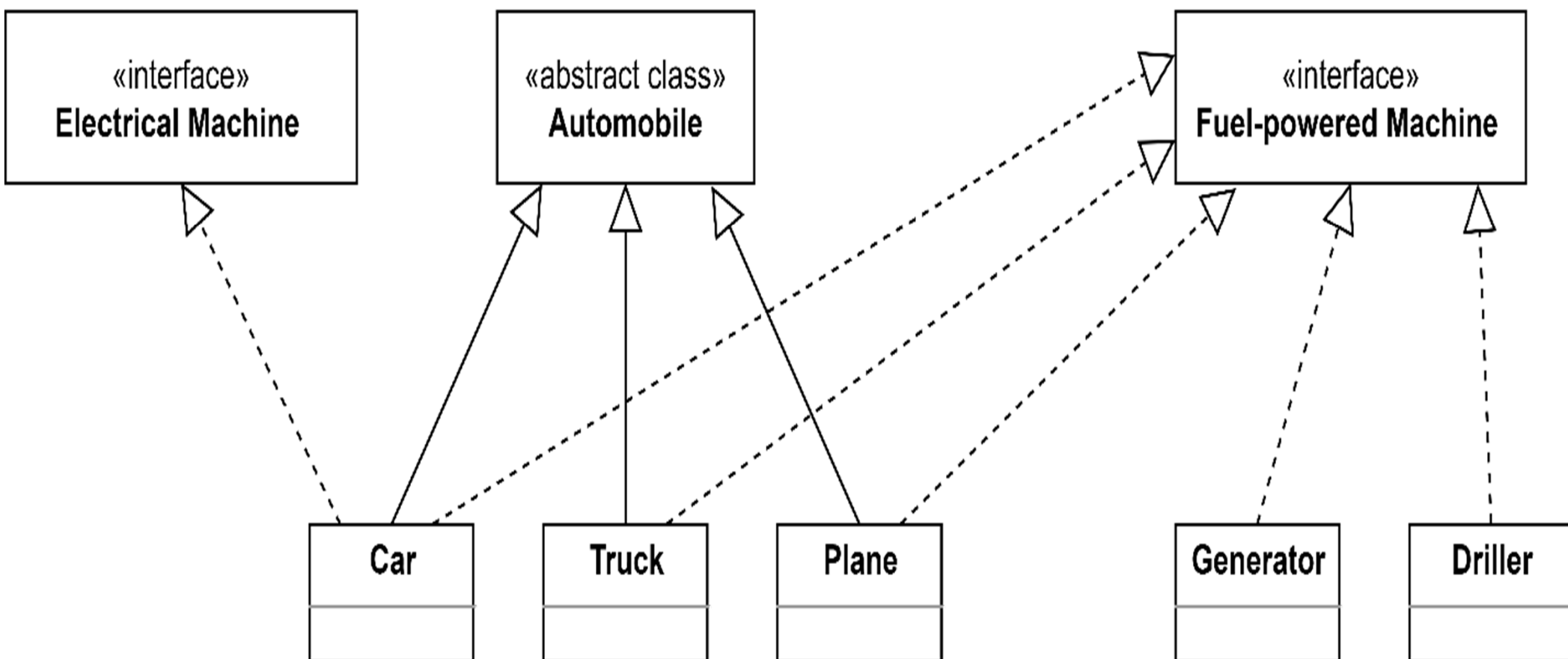
Multiple inheritance from classes is not allowed because of **Data Ambiguity!!**

Multiple implementations of interfaces is allowed

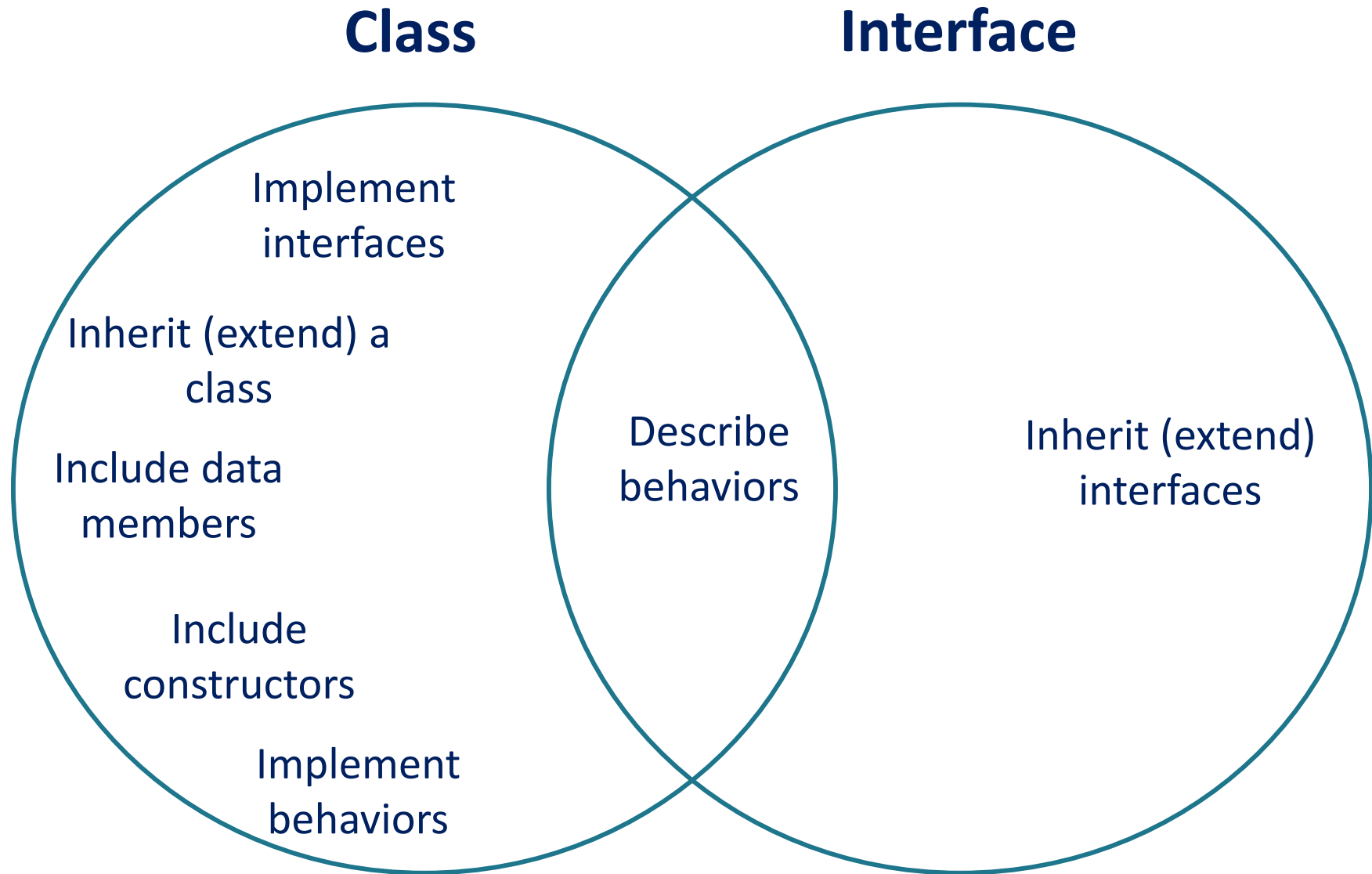
```
public interface IPublicSpeaking {  
    public void givePresentation();  
    public void speak();  
}  
  
public interface IPrivateConversation {  
    public void lowerVoiceVolume();  
    public void speak();  
}  
  
public class Person implements IPublicSpeaking, IPrivateConversation {  
    public void speak() {  
        System.out.println("This is fine");  
    }  
}
```

Implementations of the
other method
signatures are needed

Abstract Classes and Interfaces



Classes and Interfaces



References and Acknowledgments

- UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design
- Design Patterns: Elements of Reusable Object-Oriented Software
- Timeless Way of Building, Alexander, ISBN 0-19-502402-8
- Software Design and Architecture by Kenny Wong, University of Alberta Computing Science, Faculty of Science, Coursera
- A Pattern Language, Alexander, 0-19-501-919-9
- Design Patterns, Gamma, et al., 0-201-63361-2 CD version 0-201-63498-8
- Pattern-Oriented Software Architecture, Vol. 5, Buschmann, et al., 0-471-48648-5
- Concurrent Programming in Java, 2nd ed., Lea, 0-201-31009-0
- Applying UML & Patterns, 2nd ed., Larman, 0-13-092569-1
- “Object-Oriented Patterns,” P. Coad; Comm. of the ACM, 9/92
- “Documenting Frameworks using Patterns,” R. Johnson; OOPSLA '92

References and Acknowledgments

- The Art of Agile Development 1st Edition, by James Shore and Shane Warden
- Essential Scrum: A Practical Guide to the Most Popular Agile Process, Kenneth S. Rubin's Essential Scrum, 2012
- Scrum: The Art of Doing Twice the Work in Half the Time, Jeff Sutherland, 2014
- The Scrum Field Guide: Practical Advice For Your First year, Mitch Lacey, 2012
- Agile Retrospectives: Making Good Teams Great by Esther Derby & Diana Larsen
- User Story Mapping: Discover the Whole Story, Build the Right Product, 2021
- The Deadline: A Novel about Project Management
- Agile Leadership Toolkit by Peter Koning, 2019
- Akhtar, A., Bakhtawar, B. and Akhtar, S., 2022. EXTREME PROGRAMMING VS SCRUM: A COMPARISON OF AGILE MODELS. International Journal of Technology, Innovation and Management (IJTIM), 2(2).
- Ashari, I.F., Aryani, A.J. and Ardhi, A.M., 2022. Design and Build Inventory Management Information System Using the Scrum Method. JSil (Jurnal Sistem Informasi), 9(1), pp.27-35.



UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

End of Lecture